Software Architecture and Design Specification (SAD)

Project: Demand Forecasting System (DFS)
Version: 1.0
Authors: Bandugula Sai Rithesh Reddy, Dhyeya Aneesh, G Ganikhasri, Gagana C
Date: 28-10-2026
Status: Draft
Methodology: Agile (Scrum, 1-week sprints)
Architecture: Layered (3-tier) modular monolith with an asynchronous forecast worker, REST API

Team members

Name	SRN
Bandugula Sai Rithesh Reddy	PES1UG24AM064
Dhyeya Aneesh	PES1UG24AM091
G Ganikhasri	PES1UG24AM098
Gagana C	PES1UG24AM099

Revision history

Version	Date	Author	Change summary	Approval
1.0	28-10-2026	DFS Team	Initial architecture and design specification	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Dev Lead			
Product Owner			

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Demand Forecasting System (DFS) v1.0. It records the architectural decisions, the structure of the system, the forecasting pipeline and the detailed design of the main flows, interfaces and data, so that the team can build, test and maintain the system consistently.

1.2 Scope

The architecture covers all capabilities defined in the DFS SRS v1.0:

- Authentication and role-based access for Administrator, Demand Planner, Inventory Manager and Procurement Manager.
- Master data: products, raw materials, bills of materials, suppliers and material-supplier links.
- Data collection: sales-history import, inventory movements and market indicators.

- Demand forecasting with several statistical models, back-testing, automatic model selection, overrides and approval.
- Raw-material requirement planning, safety stock, reorder points and supplier-wise purchase recommendations.
- Alerts, dashboard and CSV/PDF reporting, plus cross-cutting security, performance and reliability requirements.

Out of scope for v1.0: electronic purchase-order transmission to suppliers, ERP integration through APIs and deep-learning models. The SMTP provider is treated as an external system.

1.3 Audience

Developers, QA engineers (the test plan traces to this design), security reviewers, the instructor and evaluators, and future maintainers.

1.4 Definitions

Term	Meaning
DFS	Demand Forecasting System
SRS / SAD / STP / RTM	Requirements Specification / Architecture and Design Specification / Test Plan / Requirements Traceability Matrix
SPA	Single Page Application
REST	Representational State Transfer, an HTTP/JSON API style
JWT	JSON Web Token used for stateless authentication
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper (SQLAlchemy)
ADR	Architecture Decision Record
BOM / MRP	Bill of Materials / Material Requirements Planning
MAPE	Mean Absolute Percentage Error
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

2. Document Overview

2.1 How to Use this Document

Section 3 explains the architecture: goals, stakeholders, the component view, the chosen pattern with alternatives and ADRs, technology, risks, traceability and security. Section 4 gives the detailed design: the forecasting pipeline, data model, sequence diagrams, API contracts, error handling and UX. Section 5 holds the appendices.

2.2 Related Documents

Document	Relationship
DFS SRS v1.0	Source of requirement IDs (DFS-F, DFS-NF, DFS-SR) used for traceability
DFS Test Plan v1.0	Verifies this design; test cases TC-* trace to the components defined here
RTM (SRS Section 8)	Maps each requirement to module and test case

3. Architecture

3.1 Goals and Constraints

Architectural goals

- **Correct planning data:** forecasts, requirements and purchase quantities are computed consistently from one source of truth.
- **Forecast quality:** mean MAPE <= 20% on the reference data set and better than a naive forecast (DFS-NF-003).
- **Performance:** 95% of requests within 2 s; a 500-product forecast run within 5 minutes without slowing the UI (DFS-NF-001, 002).
- **Availability:** 99.5% monthly with daily backups (DFS-NF-004).
- **Security and traceability:** TLS, hashed passwords, server-side authorisation, file validation, audit log, rate limiting (DFS-SR-001 to 006).
- **Maintainability:** at least 80% coverage enforced in CI; forecasting models pluggable behind one interface (DFS-NF-006).

Constraints

- Student team of four, Agile sprints, limited time (Sprint 0 to Sprint 5).
- Open-source tools and low-cost cloud hosting.
- Data enters through CSV/Excel files rather than a live ERP connection.
- Forecasting must be explainable to non-statisticians, so statistical models are preferred over black-box models.

3.2 Stakeholders and Concerns

Stakeholder	Concerns	Addressed by
Demand Planner	Accurate forecasts, easy data import, ability to correct the forecast with business knowledge	Forecasting Engine with model selection; override + approval workflow; import error report
Inventory Manager	Never run out of raw material; not overstock	Safety stock / reorder point calculation; Alert Service
Procurement Manager	Clear what/when/from whom to buy; respect supplier limits	MRP Service and Procurement Planner (MOQ, lead time, capacity)
Administrator	User and data management, access control	Master Data Service; RBAC in API Gateway
Management	Lower inventory cost, fewer stock-outs, visibility	Dashboard KPIs, reports
Developers	Modularity, clear boundaries, fast onboarding	Layered modules with explicit interfaces; Python end to end
QA / Test team	Testability, reproducible forecasts	Dependency injection; fixed random seeds; reference data set
Security reviewers	Protection of commercial data, traceability	Section 3.9 security architecture; audit log
Course evaluators	Traceable, justified design	Sections 3.5 and 3.8

3.3 Component (UML) Diagram

Figure 1 shows the main components and their dependencies, arranged in layers. Solid arrows show the main call direction; dashed arrows show usage dependencies and asynchronous jobs.

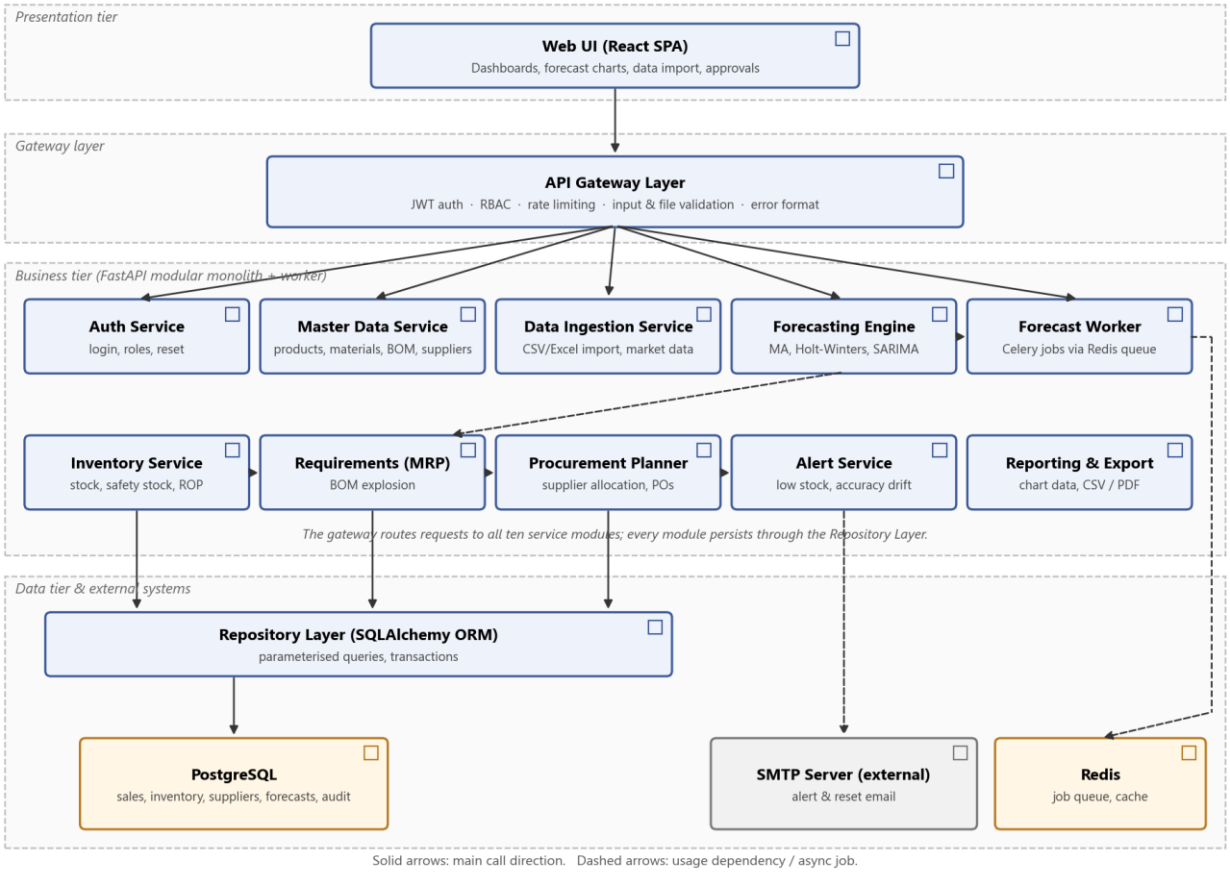


Figure 1: DFS component diagram (layered modular monolith)

3.4 Component Descriptions

Component	Responsibility	Interfaces
Web UI (React SPA)	Dashboard, master-data screens, import wizard, forecast workspace with charts and override table, purchase planning, reports, login.	Requires IApiRest
API Gateway Layer	Single entry point (FastAPI middleware and dependencies): JWT authentication, RBAC checks on every endpoint, rate limiting (HTTP 429), request and file validation, uniform error format.	Provides IApiRest; requires all service interfaces
Auth Service	Login, Argon2 password hashing, account lockout, JWT issue, password-reset tokens, user and role management.	Provides IAuth; requires IRepository, INotify
Master Data Service	Products, raw materials, BOM lines, suppliers and material-supplier links with validation.	Provides IMasterData; requires IRepository
Data Ingestion Service	Parses CSV/XLSX sales files, validates rows, upserts sales history, produces the error report; stores market indicators.	Provides IIngest; requires IMasterData, IRepository
Inventory Service	Stock movements, on-hand balance, demand variability, safety stock and reorder point.	Provides IInventory; requires IRepository
Forecasting Engine	Creates forecast runs, prepares time series, model library (moving average, Holt-Winters, SARIMAX), back-test and model selection, versions and overrides.	Provides IForecast; requires IRepository, job queue
Forecast Worker	Celery worker process that executes forecast jobs taken from Redis, so long CPU work never blocks API requests.	Consumes jobs; uses Forecasting Engine code
Requirements (MRP) Service	Explodes the approved forecast through the BOM; nets against stock and open orders.	Provides IRequirements; requires IForecast, IInventory, IMasterData
Procurement Planner	Turns net requirements into order quantities and dates and splits them across suppliers; approval workflow and draft POs.	Provides IProcurement; requires IRequirements, IMasterData

Component	Responsibility	Interfaces
Alert Service	Checks reorder points, stock-out risk and accuracy drift; dashboard notifications and email.	Provides INotify; requires IInventory, SMTP
Reporting & Export	KPI queries for the dashboard, chart data, CSV and PDF generation with formula-injection protection.	Provides IReport; requires IForecast, IProcurement
Repository Layer	SQLAlchemy data access, transactions, parameterised queries only, append-only audit log.	Provides IRepository; requires SQL
PostgreSQL	Relational store for all business data, forecasts and the audit log.	SQL over TLS
Redis	Job queue for forecast runs and short-lived cache for dashboard queries.	Redis protocol (private network)
SMTP (external)	Email delivery for alerts and password reset.	SMTP/STARTTLS

3.5 Chosen Architecture Pattern and Rationale

Layered (3-tier) client-server architecture implemented as a modular monolith, with an asynchronous worker for forecasting. A React single-page application (presentation tier) talks over HTTPS/REST to one FastAPI application (business tier) made of independent domain modules behind a gateway layer. Forecast runs, which are CPU-heavy and can take minutes, are put on a Redis queue and executed by a separate Celery worker process that uses the same Forecasting Engine code. A repository layer isolates the single PostgreSQL database (data tier). Inside the Forecasting Engine, the steps run as a pipeline (load data, clean, back-test, select, forecast, store), so new models can be added without touching the rest of the system.

Criterion	Layered modular monolith + worker (chosen)	Microservices	Plain client-server (no layering, synchronous)
Fit to scope	Good: ten well-bounded modules, one team	Over-engineered for a four-person team	Fits at first, but logic mixes with UI and data code
Performance (2 s pages, 5 min runs)	Met: forecasts run in the worker; API stays fast	Met, but adds network hops and complexity	Not met: long forecasts would block requests and time out
Data consistency	Single ACID database; forecast, BOM and stock read in one transaction	Distributed data; needs sagas / eventual consistency	Single DB, consistent
Security	One enforcement point (gateway) for authN/authZ and validation	Many service boundaries to secure	Rules scattered, easy to miss
Testability / 80% coverage	Modules unit-tested with mocked repositories; one integration environment	Many pipelines and contract tests	Hard to unit test
Operations / cost	Four containers (web, API, worker, DB) with Docker Compose on one VM	Orchestration (Kubernetes), many deployments	One deployable
Verdict	**Selected**	Rejected: complexity and cost outweigh benefits	Rejected: cannot meet DFS-NF-001/002

Why this pattern:

- **Right-sized:** the workload is a few dozen planners and managers plus periodic batch forecasts; one deployable unit is enough and is cheap to host.
- **Long jobs out of the request path:** the worker keeps the UI responsive while forecasts run (DFS-NF-001, 002), and more workers can be added if runs get bigger.
- **Strong consistency:** MRP needs the approved forecast, BOM, stock and open orders at the same moment; a single relational database gives this easily.
- **Single security choke point:** every request passes the gateway for authentication, role checks, rate limiting and validation (DFS-SR-002 to 006).
- **Evolvable:** module boundaries and the model interface allow the Forecasting Engine to be extracted as a separate service, or new models (Prophet, gradient boosting, LSTM) to be added later.

Architecture Decision Records (ADRs):

ADR	Decision	Reason	Consequence
ADR-1	Layered modular monolith over microservices	Team size, scope, hosting cost	Single deployment; module boundaries must be kept disciplined
ADR-2	Python (FastAPI) for the back end	The best forecasting libraries (pandas, statsmodels, scikit-learn) are Python; one language for API and models	Front end and back end use different languages (TypeScript / Python)
ADR-3	Asynchronous forecast runs with Celery + Redis	Runs take minutes; must not block API workers (DFS-NF-001, 002)	Extra Redis container; UI polls run status
ADR-4	Statistical models (moving average, Holt-Winters, SARIMAX) instead of deep learning in v1.0	Small data sets (monthly history), explainable to planners, fast to train	May miss complex non-linear patterns; revisit after v1.0
ADR-5	Automatic model selection by back-test MAPE per product	No single model is best for every product; objective and repeatable	Back-testing multiplies compute time by the number of models
ADR-6	Forecast versions are immutable; overrides stored as separate adjustment rows	Keeps system forecast and human change side by side; supports audit (DFS-SR-005)	Slightly more complex queries for the final value
ADR-7	PostgreSQL (relational)	Structured data with many relations (BOM, suppliers), ACID transactions, strong aggregation	Schema migrations required (Alembic)

3.6 Technology Stack and Data Stores

Concern	Choice	Notes
Front end	React 18 + TypeScript, Vite, React Router, Recharts	Responsive layout, WCAG 2.1 oriented
Back end	Python 3.12, FastAPI, Pydantic	Dependency injection, request validation, OpenAPI docs
Forecasting	pandas, NumPy, statsmodels (ExponentialSmoothing, SARIMAX), scikit-learn (metrics)	Fixed random seeds for reproducible tests
Background jobs	Celery 5 with Redis 7 as broker	Retries and time limits per job
Database	PostgreSQL 16	Daily backups; indexes on (product_id, period)
Data access	SQLAlchemy 2 + Alembic migrations	Parameterised queries only
Auth	JWT (PyJWT), Argon2 (argon2-cffi), RBAC dependencies	Short-lived access token in HTTP-only Secure cookie
Files and reports	openpyxl (Excel import), ReportLab (PDF), csv module	Server-side generation
Email	SMTP via smtplib / FastAPI-Mail; Mailpit in test	STARTTLS to provider
Testing / CI	pytest, pytest-cov, Vitest, Playwright, Postman/Newman, Locust, OWASP ZAP, Bandit, SonarCloud, GitHub Actions	As per Test Plan; coverage gate 80%
Hosting	Docker Compose on a cloud VM; Nginx reverse proxy with Let's Encrypt TLS	HTTP redirected to HTTPS; /health endpoint for uptime monitor

3.7 Risks and Mitigations

Risk	Mitigation
Poor or missing historical data (gaps, outliers, wrong codes)	Row-level validation and error report on import; gap filling and outlier capping in the pipeline; minimum-history check with moving-average fallback (DFS-F-014)
Forecasts become inaccurate after market changes	Market indicators as model inputs; planner overrides; accuracy-drift alert when MAPE exceeds threshold (DFS-F-019)
Forecast runs too slow for many products	Background worker; runs parallelised per product; more workers can be added; time limit per job

Risk	Mitigation
Supplier data (lead time, price) out of date	Last-updated date shown on supplier screen; recommendations show the data they used
Leak of commercially sensitive prices and sales	RBAC, TLS, owner/role checks on exports, audit log
Team unfamiliar with time-series models	Sprint 0 spike on statsmodels; reference data set with known answers; pair programming
Low-cost hosting downtime	Uptime monitor, daily backups, documented redeploy with Docker Compose

3.8 Traceability to Requirements

Req ID	Requirement	Component(s)	Test case
DFS-F-001	Login and session timeout	Auth Service, Web UI	TC-Auth-01
DFS-F-002	Role-based access	API Gateway (RBAC), Auth Service	TC-Auth-02
DFS-F-003	Password reset	Auth Service, Alert Service (email)	TC-Auth-03
DFS-F-004	Products, materials and BOM	Master Data Service	TC-MD-01
DFS-F-005	Supplier management	Master Data Service	TC-MD-02
DFS-F-006	Material-supplier mapping	Master Data Service	TC-MD-03
DFS-F-007	Import sales history	Data Ingestion Service, API Gateway (file checks)	TC-Data-01
DFS-F-008	Inventory and stock movements	Inventory Service	TC-Data-02
DFS-F-009	Market indicators	Data Ingestion Service	TC-Data-03
DFS-F-010	Generate forecast	Forecasting Engine, Forecast Worker	TC-Fc-01
DFS-F-011	Back-test and best-model selection	Forecasting Engine (model selector)	TC-Fc-02
DFS-F-012	Forecast chart and intervals	Web UI, Reporting & Export	TC-Fc-03
DFS-F-013	Forecast override and approval	Forecasting Engine (versions, adjustments), audit log	TC-Fc-04
DFS-F-014	Short-history fallback	Forecasting Engine (fallback rule)	TC-Fc-05
DFS-F-015	BOM explosion (MRP)	Requirements (MRP) Service	TC-Proc-01
DFS-F-016	Safety stock and reorder point	Inventory Service	TC-Inv-01
DFS-F-017	Purchase recommendations	Procurement Planner	TC-Proc-02
DFS-F-018	Approve recommendation / draft PO	Procurement Planner	TC-Proc-03
DFS-F-019	Low-stock and accuracy alerts	Alert Service	TC-Inv-02
DFS-F-020	Dashboard KPIs	Reporting & Export, Web UI	TC-Rep-01
DFS-F-021	CSV / PDF export	Reporting & Export	TC-Rep-02
DFS-NF-001	Response time	Whole stack; indexes, caching	TC-Perf-01
DFS-NF-002	Forecast run time	Forecast Worker (Celery), Redis	TC-Perf-02
DFS-NF-003	Forecast accuracy	Forecasting Engine	TC-Fc-06
DFS-NF-004	Availability and backup	Deployment, monitoring, backups	TC-Rel-01
DFS-NF-005	Responsive and accessible UI	Web UI	TC-UX-01
DFS-NF-006	80% test coverage	Modular design, CI gate	CI coverage report
DFS-NF-007	Data volume	PostgreSQL indexes, partitioning	TC-Perf-03
DFS-SR-001	TLS 1.2+	Nginx reverse proxy	TC-Sec-01
DFS-SR-002	Password hashing and lockout	Auth Service (Argon2)	TC-Sec-02
DFS-SR-003	Server-side authorisation	API Gateway	TC-Sec-03
DFS-SR-004	Input and file validation	API Gateway, Ingestion, Reporting	TC-Sec-04
DFS-SR-005	Audit log	Audit log (Repository Layer)	TC-Sec-05

Req ID	Requirement	Component(s)	Test case
DFS-SR-006	Rate limiting	API Gateway	TC-Sec-06

3.9 Security Architecture

Security is enforced in layers (defence in depth), with the API gateway layer as the single enforcement point for every inbound request.

Threat model (STRIDE)

Threat	Example in DFS	Mitigation
Spoofing	Stolen or guessed password; forged token	Argon2 hashes, lockout after 5 failures, signed short-lived JWT in HTTP-only Secure cookie, single-use reset tokens (DFS-SR-002)
Tampering	Changing forecast numbers or supplier prices without permission; uploading a crafted file; SQL injection	Server-side RBAC; approved versions read-only; file type/size/content checks; parameterised queries (DFS-SR-003, 004)
Repudiation	A planner denies changing a forecast; a manager denies approving a large purchase	Append-only audit log with user, time, old and new value (DFS-SR-005)
Information disclosure	Competitor learns supplier prices or sales volumes; sniffing traffic	TLS 1.2+; role-based access to prices and exports; no sensitive data in logs (DFS-SR-001, 003)
Denial of service	Flooding login, or starting many huge forecast runs	Rate limiting (HTTP 429); one active run per scope; job time limits; request size limits (DFS-SR-006)
Elevation of privilege	Inventory Manager calls the purchase-approval API	Deny-by-default role checks on every endpoint; tests using other roles' tokens (DFS-SR-003)

Other security design points

- **XSS:** React output-encodes all text; user-entered HTML is never rendered; Content-Security-Policy and security headers set at Nginx.
- **CSRF:** SameSite=Strict cookies plus an anti-CSRF token on state-changing requests.
- **Spreadsheet formula injection:** exported cells beginning with =, +, - or @ are prefixed with a quote so they open as text.
- **Secrets** (database password, JWT key, SMTP credentials) are kept in environment variables, never in source control.
- **Security objectives:** SO-1 Confidentiality (SR-001, 002, 003, 006); SO-2 Integrity (SR-003, 004); SO-3 Accountability (SR-005).

4. Design

4.1 Design Overview

The back end is organised as feature modules, each with the same internal structure: Router (HTTP boundary and Pydantic validation), Service (business rules) and Repository (persistence). Modules call each other only through service interfaces; routers never use another module's repository. Common concerns (authentication, role checks, rate limiting, exception handling, request logging) live in the shared gateway layer. Dependencies are injected, so services can be unit-tested with mocked repositories.

Forecasting pipeline

- **1. Load:** read monthly (or weekly) demand per product and the market indicators for the same periods.
- **2. Clean:** fill missing periods with zero demand, cap outliers at 3 standard deviations, check minimum history (fewer than 12 periods means moving-average fallback, flagged Low confidence).

- **3. Back-test:** for each candidate model (moving average, Holt-Winters, SARIMAX with indicators), train on all but the last 3 periods, forecast those 3, and compute MAPE, MAE and bias.
- **4. Select:** choose the model with the lowest MAPE for that product.
- **5. Forecast:** retrain the selected model on the full history and forecast the requested horizon with 80% and 95% prediction intervals.
- **6. Store:** save the forecast lines, metrics and chosen model as a new forecast version with status DRAFT.

All models implement one interface, ForecastModel.fit(series, exog) and ForecastModel.predict(horizon, exog), so a new model is added by writing one class and registering it.

Planning calculations

- **Gross requirement** of material m in period t = sum over products p of (approved forecast of p in t x BOM quantity of m in p).
- **Net requirement** = gross requirement - projected on-hand stock - open purchase-order quantity due by t (never below zero).
- **Safety stock** = z x sigma(daily demand) x sqrt(lead time in days), with z = 1.65 for a 95% service level.
- **Reorder point** = average daily demand x lead time + safety stock.
- **Supplier split:** fill from the preferred supplier up to its monthly capacity, then the next supplier ranked by a score of price and reliability; round each line up to the supplier's MOQ; order-by date = need date - supplier lead time.

Lifecycles

Forecast run: QUEUED -> RUNNING -> DONE or FAILED. **Forecast version:** DRAFT (overrides allowed) -> APPROVED (read-only; used by MRP) -> SUPERSEDED when a newer version is approved. **Purchase recommendation:** PENDING -> APPROVED (draft POs created) or REJECTED (comment required).

Key data entities

Entity	Main attributes
user	id, email, password_hash, role, failed_logins, locked_until
product / raw_material	id, code, name, unit, category, active
bom_line	product_id, material_id, qty_per_unit
supplier / material_supplier	id, name, lead_time_days, moq, unit_price, monthly_capacity, reliability_pct / material_id, supplier_id, preferred
sales_history	product_id, period, quantity, region (unique on product_id, period, region)
market_indicator	product_id or category, period, promotion, price_change_pct, holiday_flag, trend_index
stock_movement	item_id, type (receipt/issue/adjustment), qty, date, reference
forecast_run / forecast_version / forecast_line	run status, parameters / version status, approved_by / product_id, period, value, lower80, upper80, lower95, upper95, model, mape
forecast_adjustment	line_id, old_value, new_value, reason, user_id, timestamp
recommendation / recommendation_line / purchase_order	status, horizon / material_id, supplier_id, qty, order_by, need_by, unit_price / supplier_id, lines, status DRAFT
audit_log	id, user_id, action, entity, entity_id, old_value, new_value, timestamp (append-only)

4.2 UML Sequence Diagrams

4.2.1 Running a demand forecast

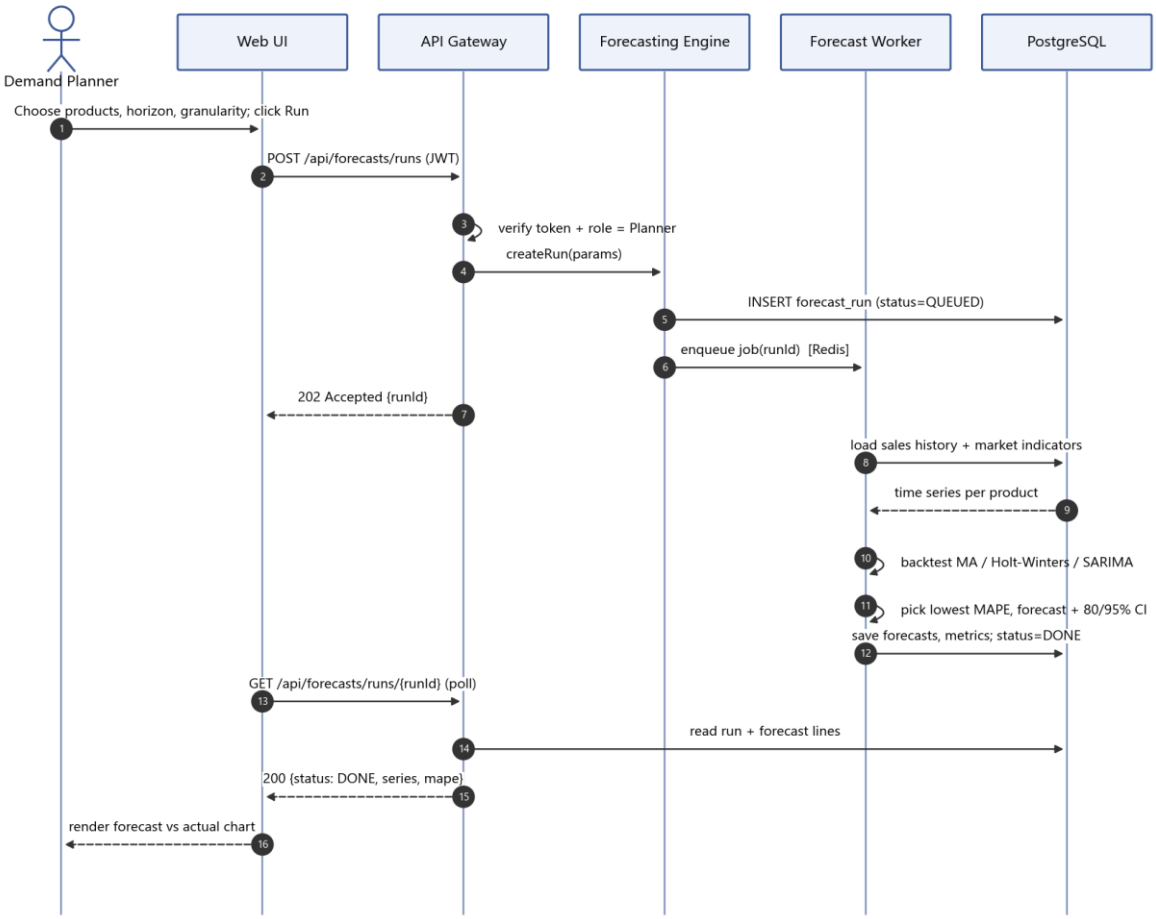


Figure 2: Forecast run flow (DFS-F-010, 011, 012, NF-002)

4.2.2 Generating and approving a purchase recommendation

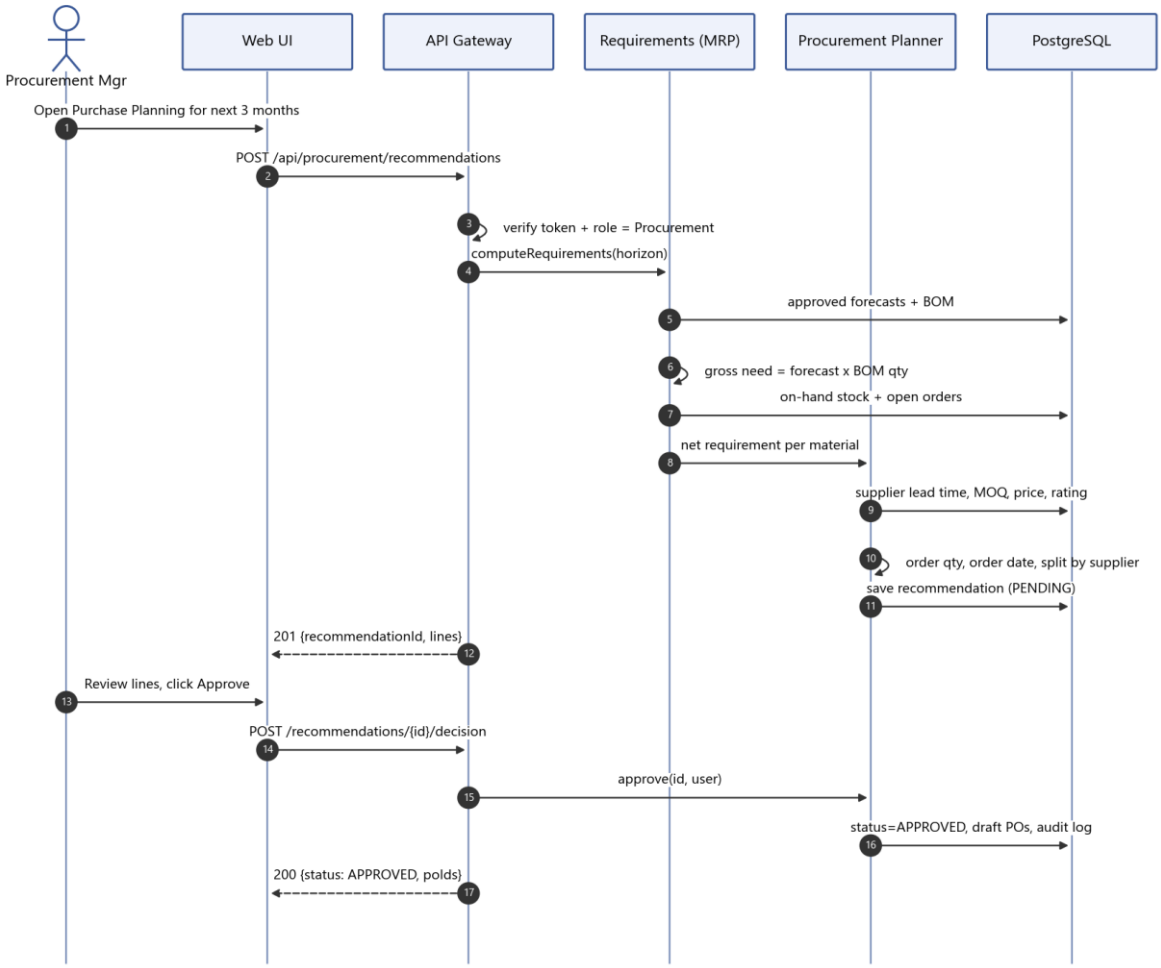


Figure 3: Purchase recommendation flow (DFS-F-015, 017, 018, SR-003, SR-005)

4.2.3 Importing sales data and raising alerts

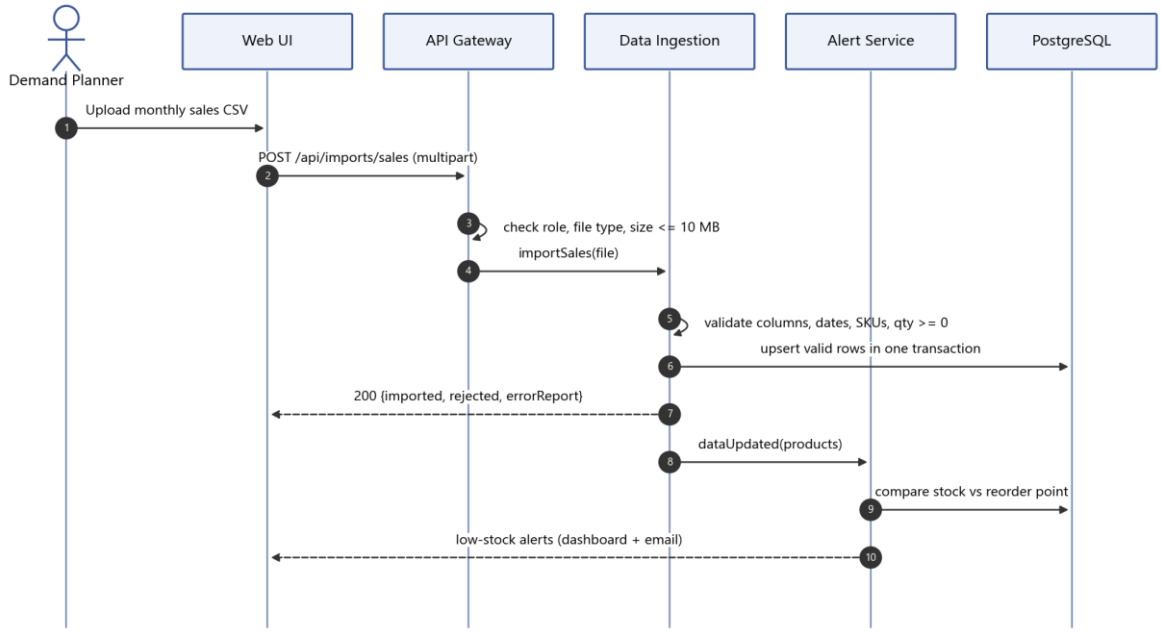


Figure 4: Sales import and alert flow (DFS-F-007, 019, SR-004)

4.3 API Design

Interfaces are defined for three components (REST over HTTPS, JSON, base path /api). All endpoints require authentication and a role check.

Interface 1: Forecasting Engine - start and read a forecast run

Item	Definition
Endpoint	POST /api/forecasts/runs (role: Demand Planner)
Request	{ productIds: [..] | "ALL", horizon: 6, granularity: "MONTH" | "WEEK", models?: ["MA", "HW", "SARIMAX"], useMarketIndicators: true }
Success	202 { runId, status: "QUEUED" }
Errors	400 invalid horizon or granularity; 401 not authenticated; 403 wrong role; 409 a run for the same products is already running
Endpoint	GET /api/forecasts/runs/{runId}
Success	200 { runId, status, versionId, products: [{ productId, model, mape, lowConfidence, points: [{ period, forecast, lower80, upper80, lower95, upper95 }] }] }
Errors	401 / 403; 404 unknown run

Interface 2: Data Ingestion - import sales history

Item	Definition
Endpoint	POST /api/imports/sales (multipart/form-data, role: Demand Planner)
Request	file: .csv or .xlsx with columns date, product_code, quantity, region (optional)
Success	200 { imported: 1180, updated: 20, rejected: 5, errors: [{ row: 42, field: "product_code", message: "Unknown product" }] }
Errors	400 missing columns; 401 / 403; 413 file larger than 10 MB; 415 unsupported file type

Interface 3: Procurement Planner - recommendations

Item	Definition
Endpoint	POST /api/procurement/recommendations (role: Procurement Manager)
Request	{ forecastVersionId, horizonMonths: 3 }
Success	201 { recommendationId, status: "PENDING", lines: [{ materialId, supplierId, quantity, orderBy, needBy, unitPrice, totalCost }], warnings: [..] }
Errors	401 / 403; 404 version not found; 422 forecast version not approved or a material has no active supplier
Endpoint	POST /api/procurement/recommendations/{id}/decision
Request	{ decision: "APPROVE" | "REJECT", comment?, lineEdits?: [{ lineId, quantity }] }
Success	200 { status: "APPROVED", purchaseOrderIds: [..] } or 200 { status: "REJECTED" }
Errors	400 reject without comment or quantity below MOQ; 401 / 403; 409 already decided

Standard error format

All errors use { error: { code, message, details? }, requestId }. Messages never reveal stack traces or internal details, and login and password reset return the same generic message whether or not the email exists.

4.4 Error Handling, Logging and Monitoring

- **Error handling:** a global exception handler maps domain errors to HTTP codes (400, 401, 403, 404, 409, 413, 415, 422, 429, 500) and the standard error format. Database writes run in transactions and roll back on failure; an import is all-or-nothing for valid rows.
- **Forecast job failures:** a model that fails to converge for a product is skipped and the next-best model is used; if all fail, the product falls back to moving average. A job that crashes is retried once, then marked FAILED with a reason shown to the planner.
- **Logging:** structured JSON logs with request ID, user ID, endpoint, status and latency. Passwords, tokens and file contents are never logged. Security events (failed logins, 403s, 429s, approvals) also go to the audit log.
- **Monitoring:** uptime monitor on /health (supports the 99.5% target), API response-time percentiles, error rate, forecast run duration and failure count, queue length, and alerts when thresholds are crossed.
- **Resilience:** database connection pool and timeouts; retry with back-off for SMTP; daily backups with a tested restore.

4.5 UX Design

- Role-based home dashboard showing only what that role needs (e.g. the Inventory Manager sees low-stock alerts first).
- Import wizard: upload, preview first rows, validate, then a summary with a downloadable error report.
- Forecast workspace: product picker, chart of actual vs forecast with shaded 80%/95% intervals, model name and MAPE badge, 'Low confidence' tag, editable table for overrides with a reason box, and an Approve button.
- Purchase planning: table of materials with need date, quantity, supplier split and cost; inline edit; Approve/Reject with comment.
- Accessibility (WCAG 2.1 AA): keyboard navigation, visible focus, labels, sufficient contrast, colour not used alone, every chart paired with a data table.
- Plain-language messages, e.g. 'Order 1,200 kg of steel from Supplier A by 12-Jan to avoid a stock-out on 26-Jan'.

4.6 Open Issues and Next Steps

- Evaluate Prophet and gradient-boosting / LSTM models once more data is available; they plug into the same model interface.
- Direct ERP integration (REST or database connector) instead of file import.
- Sending approved purchase orders to suppliers by email or a supplier portal.
- Multi-plant / multi-warehouse planning and currency handling.
- Extract the Forecasting Engine as a separate service if run volume grows.

5. Appendices

5.1 Glossary

See Section 1.4 and the SRS Section 1.4. **Forecast version:** a frozen set of forecast values for a run; only one version is APPROVED at a time. **Net requirement:** material still to be bought after counting stock and open orders.

5.2 References

- IEEE 42010 (Architecture description)
- DFS SRS v1.0 and DFS Test Plan v1.0
- OWASP Top 10 and OWASP ASVS
- WCAG 2.1
- NIST SP 800-160
- Hyndman, R. J. and Athanasopoulos, G., Forecasting: Principles and Practice (time-series methods and accuracy measures)

5.3 Tools

draw.io / Matplotlib (diagrams), Swagger / OpenAPI, GitHub Actions, SonarCloud, Postman/Newman, Playwright, Locust, OWASP ZAP, Mailpit, Docker Compose.